



국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

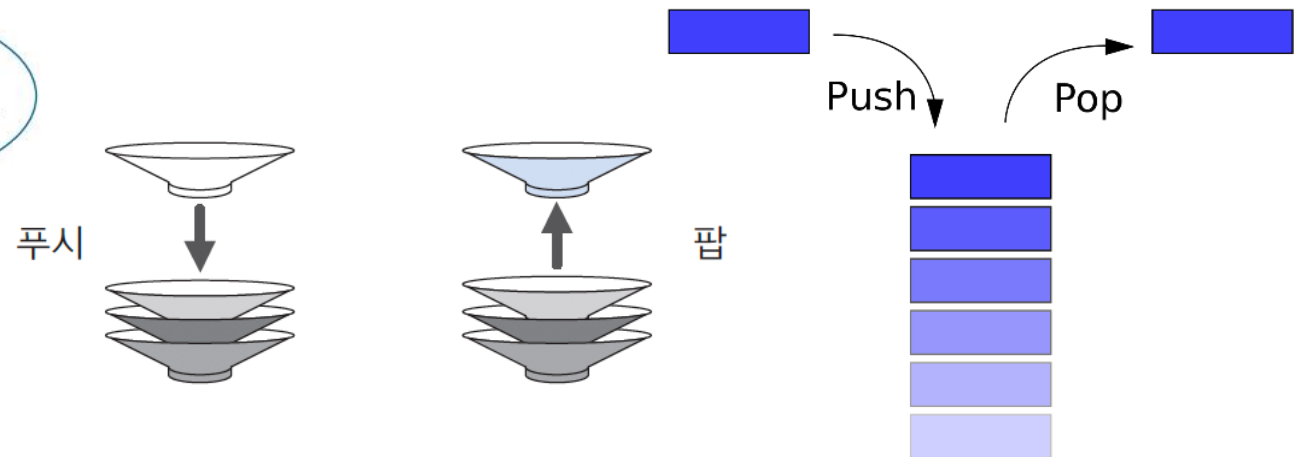
친환경스마트조선기자재학과

데이터처리특론



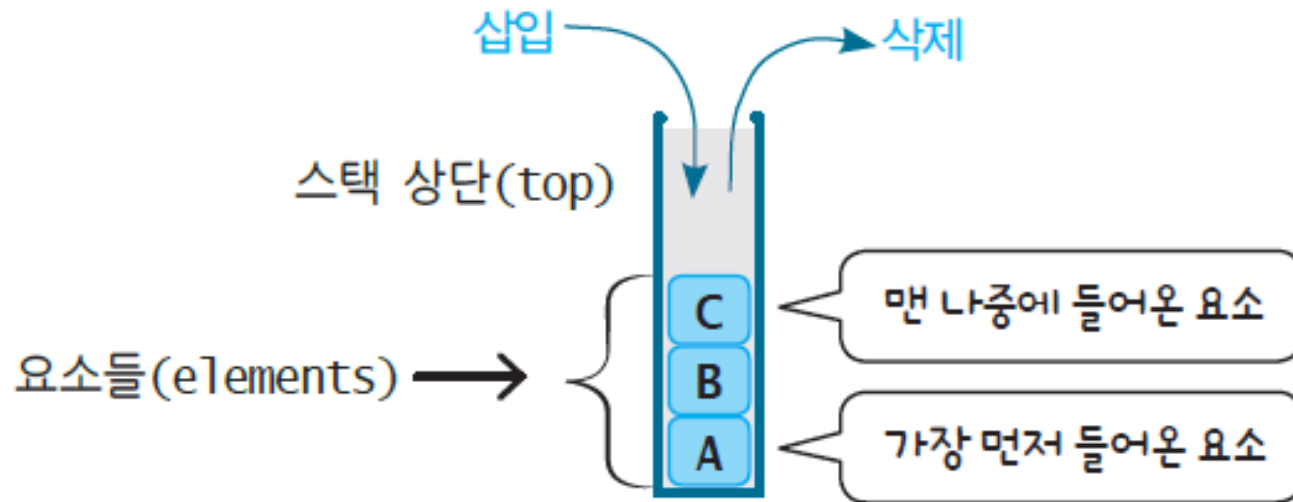
스택

- "stack": 쌓아놓은 더미
- 스택은 한 쪽 끝에서만 자료를 넣거나 뺄 수 있는 선형 구조(LIFO - Last In First Out)으로 되어 있다.
- 자료를 넣는 것을 '밀어넣는다' 하여 푸시(push)라고 하고 반대로 넣어둔 자료를 꺼내는 것을 팝(pop)이라고 하는데, 이때 꺼내지는 자료는 가장 최근에 푸시한 자료부터 나오게 된다.
- 이처럼 나중에 넣은 값이 먼저 나오는 것을 LIFO 구조라고 한다.

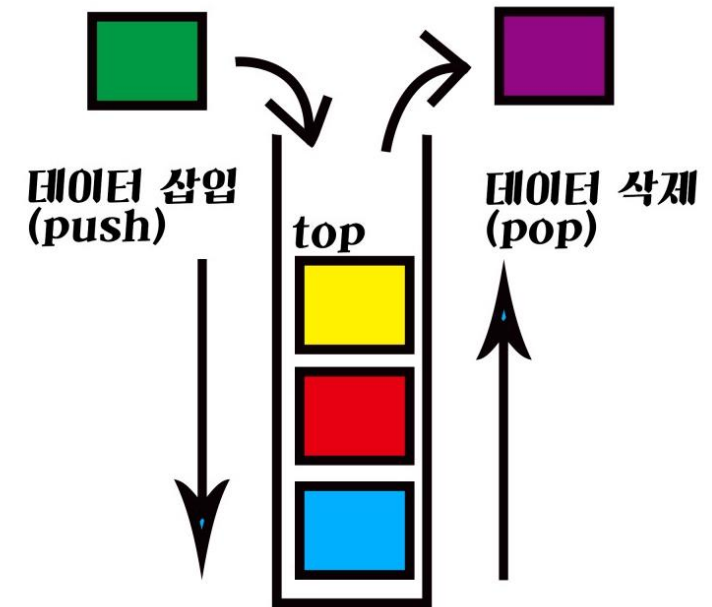


스택의 구조

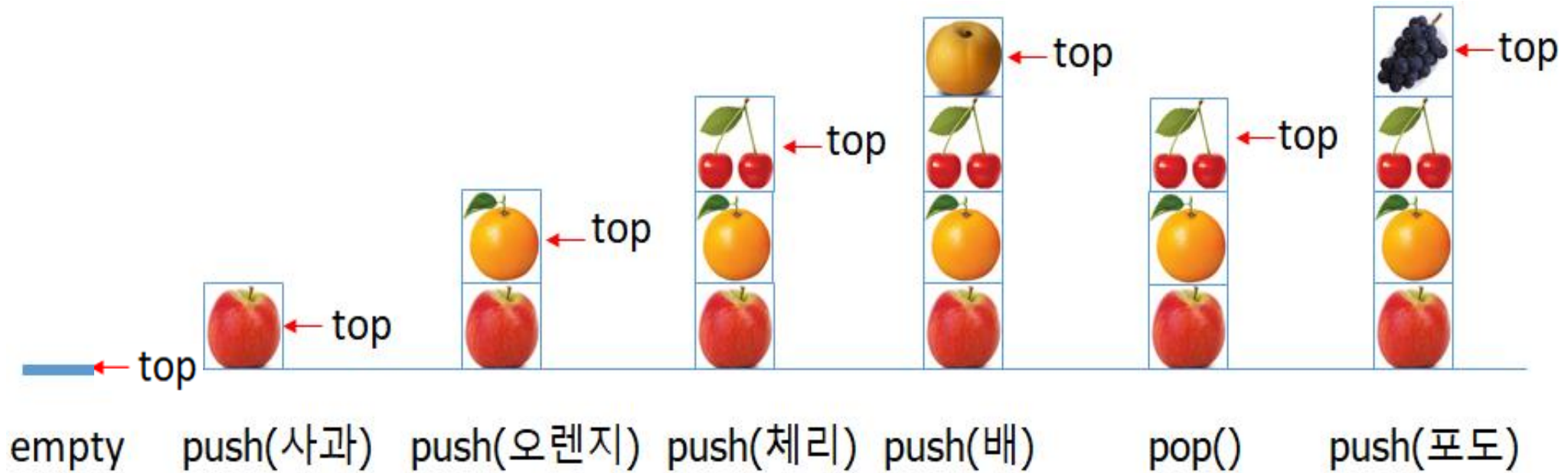
- 다른 통로들은 모두 막고 한쪽만을 열어둔 구조
- 이때 열린 곳을 보통 스택 상단(stack top)
- 스택은 요소의 삽입과 삭제가 상단에서만 가능한 자료구조



stack의 구조



스택의 PUSH와 POP 연산



추상 자료형이란?

- 프로그래밍 언어에서 다양한 자료형 제공

예) 파이썬: 정수(int), 실수(float), 문자열(str) 등

35: int, 35.0: float, "35": str

- 새로운 자료형이 필요하면?

그 자료형의 자세하고 복잡한 내용 대신에 필수적이고 중요한 특징만 골라서 단순화시키는 작업이 필요

→ 추상화(abstraction) → 추상 자료형

- 추상 자료형(ADT: Abstract Data Type)

그 자료형이 어떤 자료를 다루고, 어떤 연산이 필요한지를 정의해 보는 것 정도로 이해하면 됨

스택의 추상 자료형

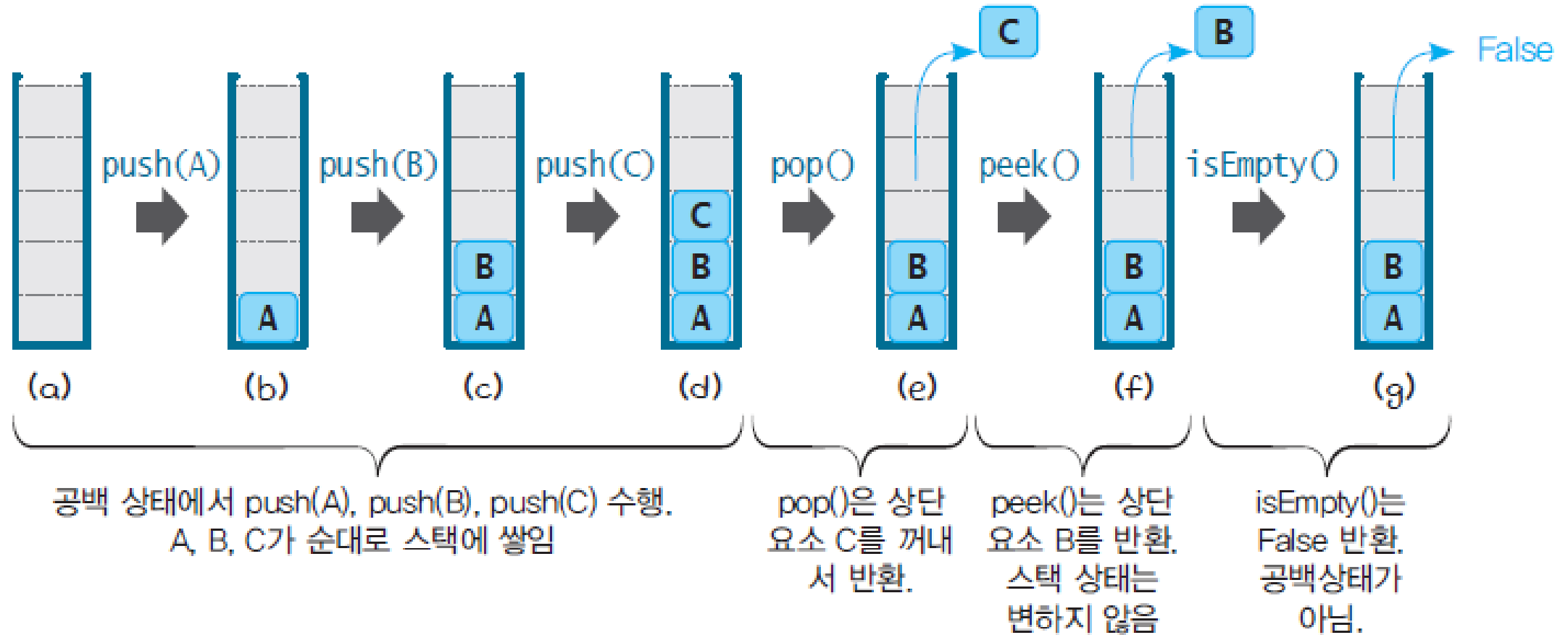
- 스택이 다루는 데이터
숫자와 문자열을 포함한 어떤 자료든 저장할 수 있음
- 스택의 연산
핵심적인 연산들만 관심있음

Stack ADT

데이터: 후입선출(LIFO)의 접근 방법을 유지하는 항목들의 모임
연산

- `Stack()`: 비어 있는 새로운 스택을 만든다.
- `isEmpty()`: 스택이 비어있으면 `True`를 아니면 `False`를 반환한다.
- `push(e)`: 항목 `e`를 스택의 맨 위에 추가한다.
- `pop()`: 스택의 맨 위에 있는 항목을 꺼내 반환한다.
- `peek()`: 스택의 맨 위에 있는 항목을 삭제하지 않고 반환한다.
- `size()`: 스택내의 모든 항목들의 개수를 반환한다.
- `clear()`: 스택을 공백상태로 만든다.

일련의 연산 예



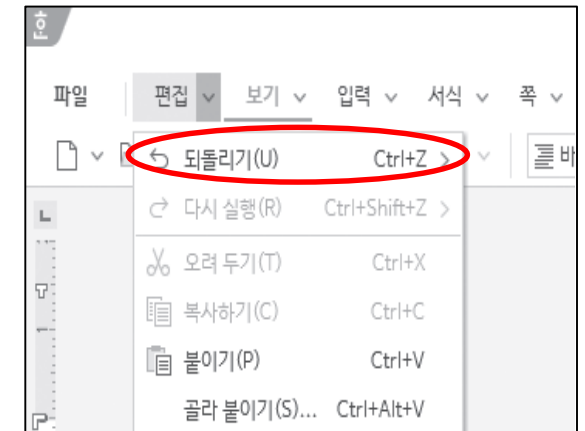
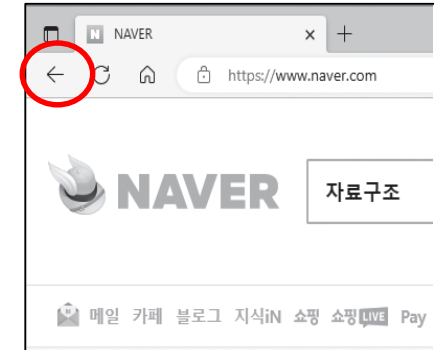
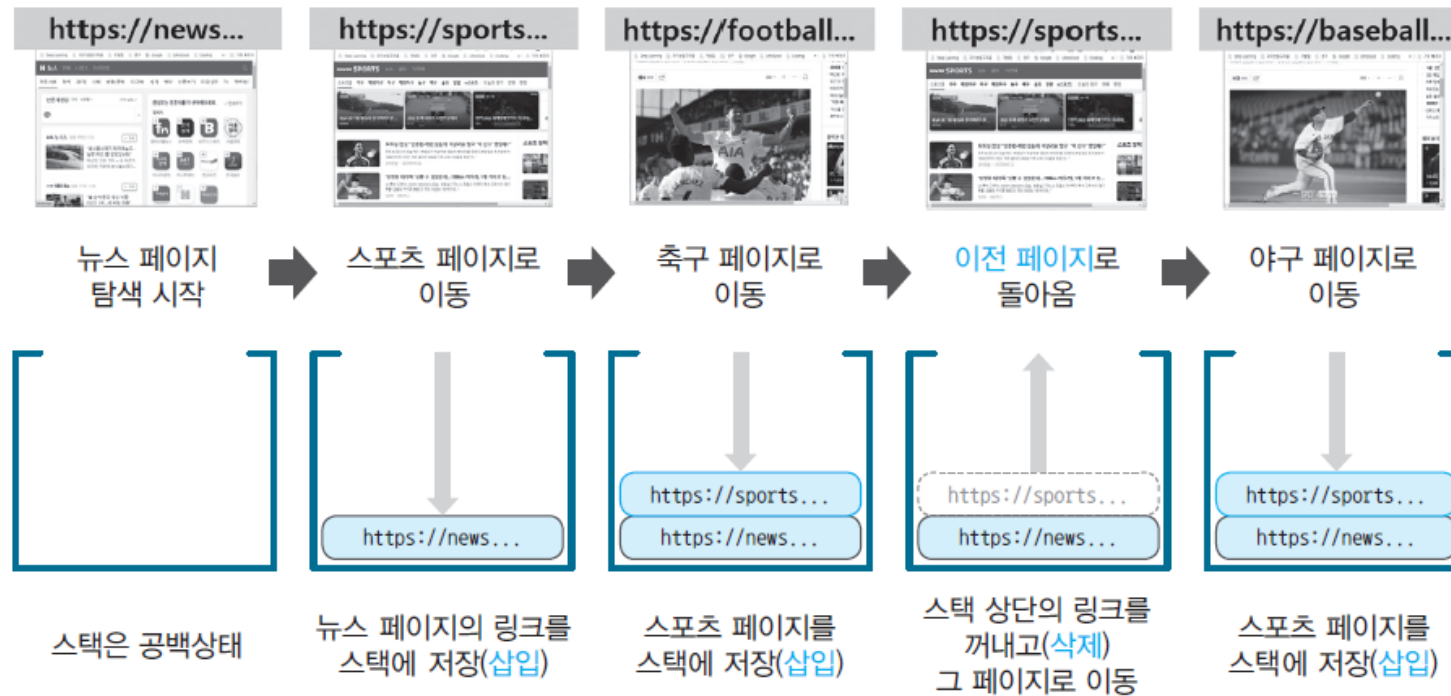
2가지 오류 상황



일반적인 언어에서 배열은 크기를 고정시키고 생성함

스택의 활용 예

- 웹 브라우저의 이전페이지로 이동 기능



- 편집기의 되돌리기
- 괄호 검사, 계산기, 미로 탐색 등

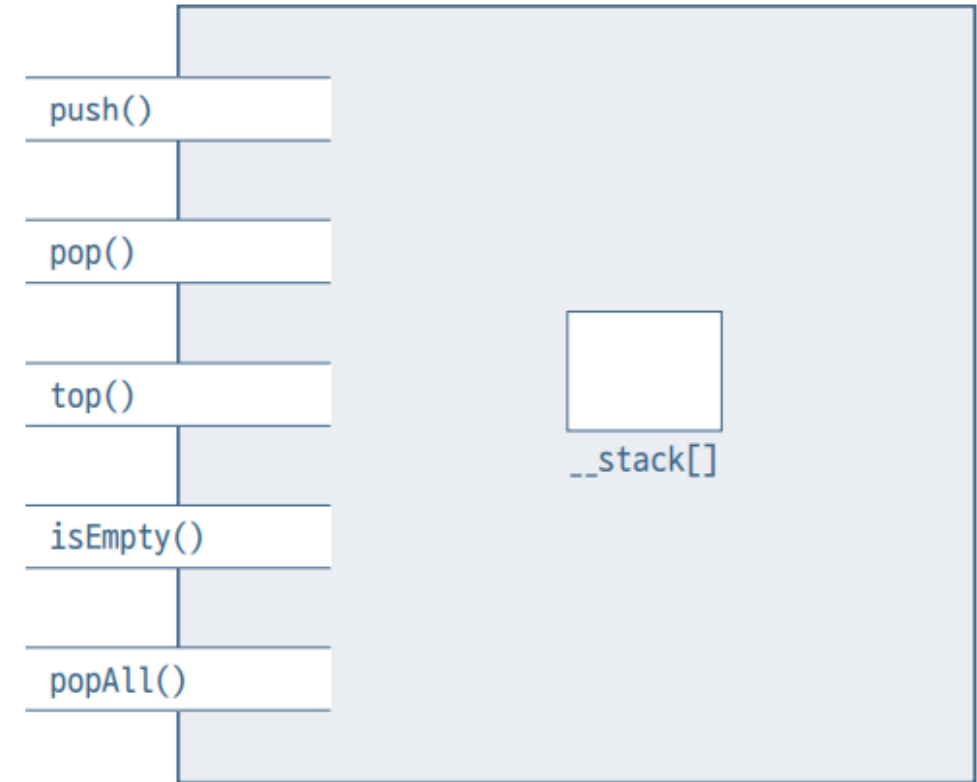
리스트 스택 객체 구조

필드:

__stack[] ◀ 스택 원소들이 저장되는 리스트

작업:

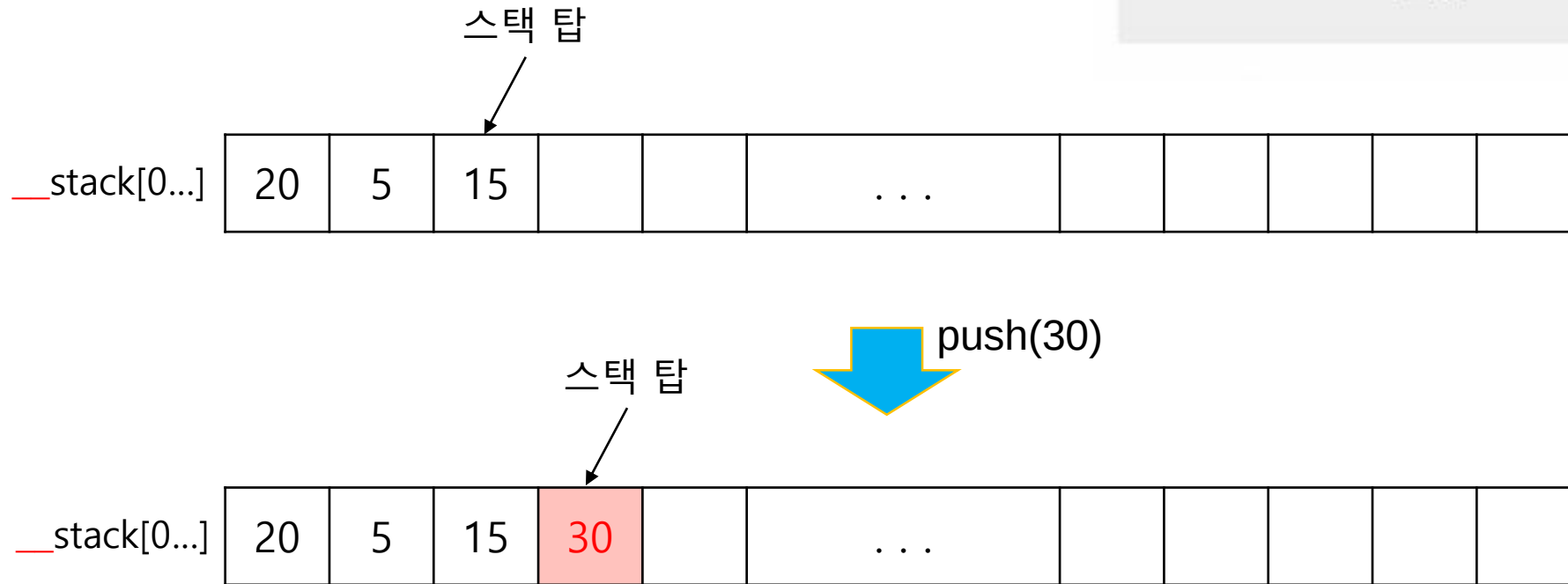
push() ◀ 스택의 맨 위에 원소를 삽입한다
pop() ◀ 스택의 맨 위에 있는 원소를 알려주고 삭제한다
top() ◀ 스택의 맨 위에 있는 원소를 알려준다
isEmpty() ◀ 스택이 빈 스택인지를 알려준다
popAll() ◀ 스택을 깨끗이 청소한다



리스트를 이용한 스택 객체 구조

삽입(insertion) : PUSH

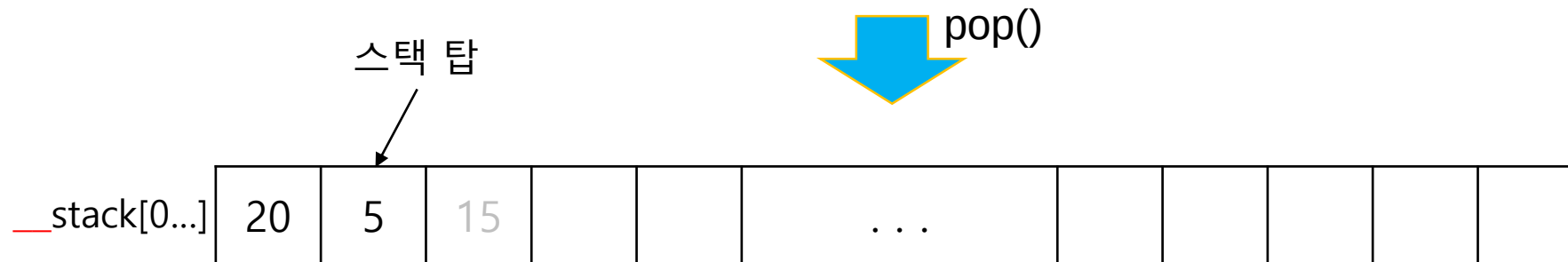
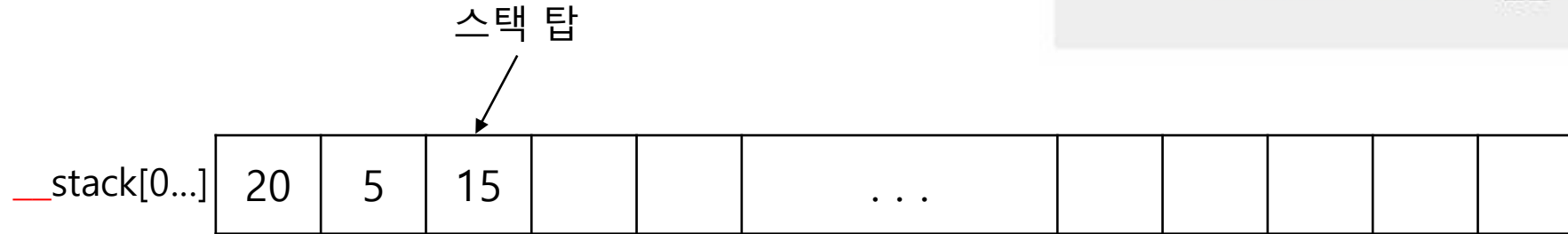
```
def push(self, x):
    self.__stack.append(x)
```



리스트 스택에 원소를 삽입하는 예

삭제(Deletion) : POP

```
def pop(self):
    return self.__stack.pop()
```



배열 스택에서 원소를 삭제하는 예

기타 작업

top() `return stack[끝 인덱스]`

isEmpty() `return len(self.__stack)==0`

또는, `return not bool(self.__stack)`

```
def isEmpty(self) -> bool:  
    return not bool(self.__stack)
```

popAll() `self.__stack.clear()`

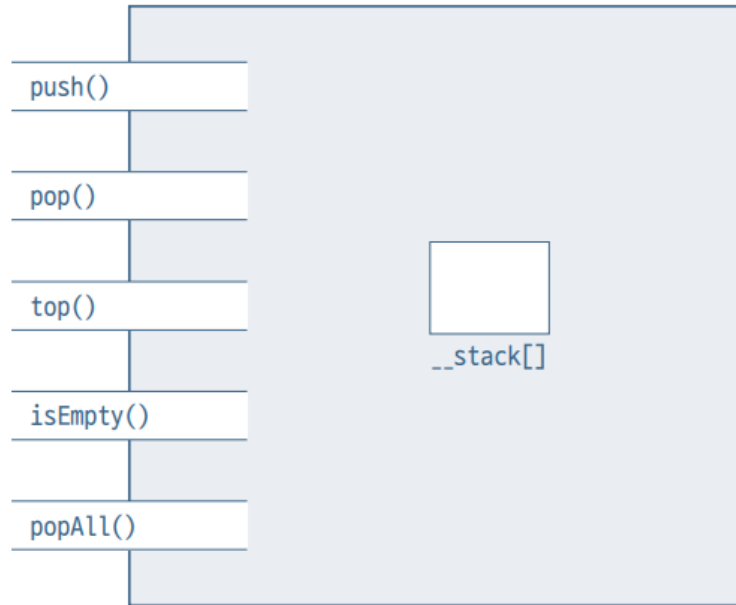
또는, `self.__stack = []`

```
def popAll(self):  
    self.__stack = []
```

리스트 스택의 클래스 구조

```
class ListStack:
    def __init__(self):
        self.__stack = []
    def push(self, x):
        ...
    def pop(self):
        ...
    def top(self):
        ...
    def isEmpty(self):
        ...
    def popAll(self):
        ...
}
```

파이썬 구현



리스트 스택 객체 구조

```

class ListStack:
    def __init__(self):
        self.__stack = []

    def push(self, x):
        self.__stack.append(x)

    def pop(self):
        return self.__stack.pop()

    def top(self):
        if self.isEmpty():
            return None
        else:
            return self.__stack[-1]

    def isEmpty(self) -> bool:
        return not bool(self.__stack)
        # 또는 return len(self.__stack) == 0

    def popAll(self):
        self.__stack.clear()

    def printStack(self):
        print("Stack from top:", end=' ')
        for i in range(len(self.__stack) - 1, -1, -1):
            print(self.__stack[i], end=' ')
        print()
    
```

클래스 ListStack 사용 예

리스트 스택 사용 예

```
st1 = ListStack()  
print(st1.top()) # No effect  
st1.push(100)  
st1.push(200)  
print("Top is", st1.top())  
st1.pop()  
st1.push('Monday')  
st1.printStack()  
print('isEmpty?', st1.isEmpty())
```

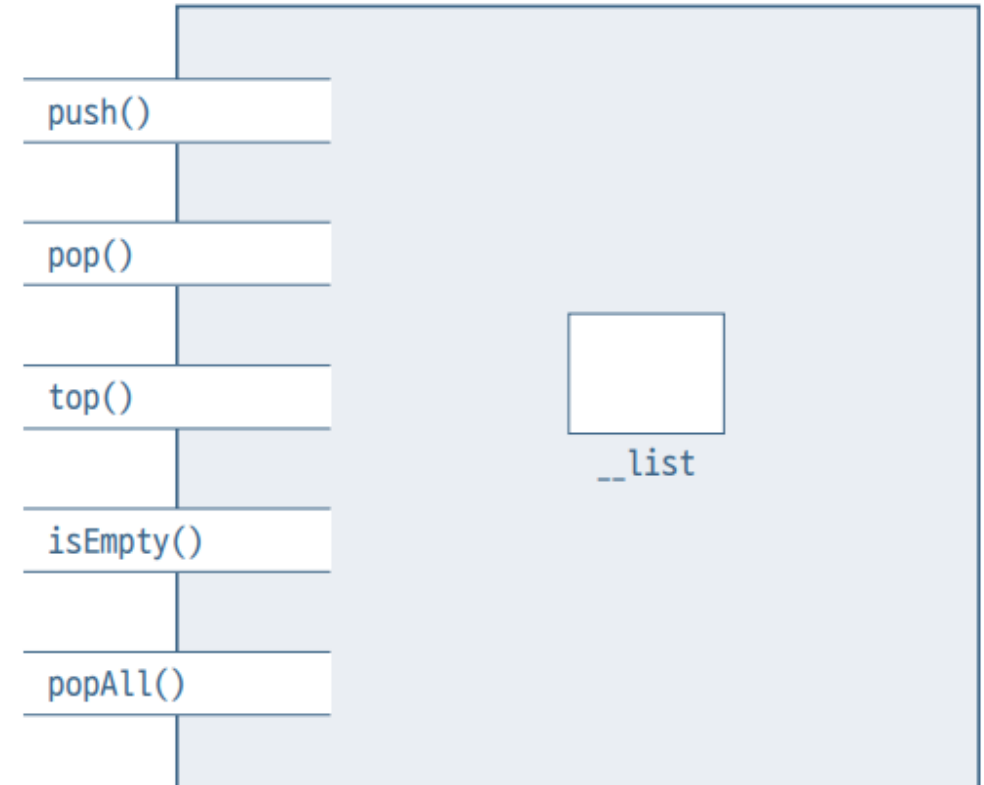
```
None  
Top is 200  
Stack from top: Monday 100  
isEmpty? False
```

수행 결과

```
Top is 200  
Stack from top: Monday 100  
isEmpty? False
```


연결 리스트 스택 객체 구조

- 필드: `__list` ◀ 스택 원소들이 저장되는 연결 리스트
- 작업:
- `push()` ◀ 스택의 맨 위에 원소를 삽입한다
 - `pop()` ◀ 스택의 맨 위에 있는 원소를 알려주고 삭제한다
 - `top()` ◀ 스택의 맨 위에 있는 원소를 알려준다
 - `isEmpty()` ◀ 스택이 빈 스택인지를 알려준다
 - `popAll()` ◀ 스택을 깨끗이 청소한다



연결 리스트 스택의 객체 구조

연결 리스트 스택 클래스 구조

```
class LinkedStack:
    def __init__(self):
        self.__list = LinkedListBasic()
    def push(self, x):
        ...
    def pop(self):
        ...
    def top(self):
        ...
    def isEmpty(self):
        ...
    def popAll(self):
        ...
    ...
```

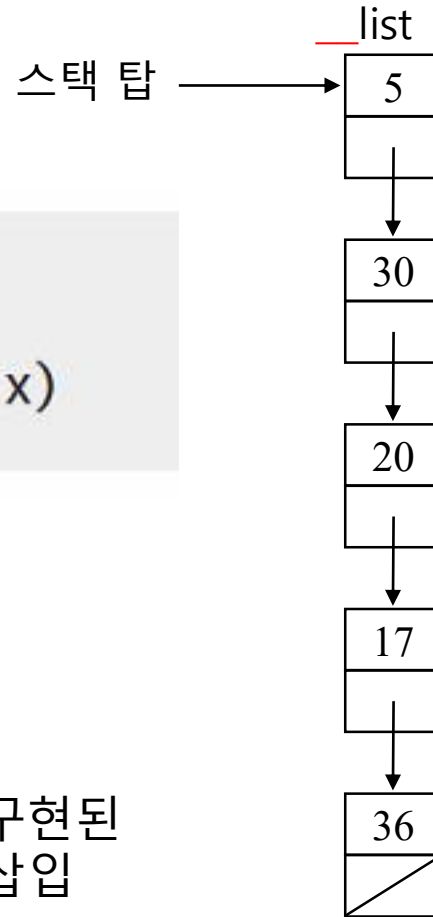
삽입(insertion) : PUSH

__list 대신 list를 문맥에 따라 혼용

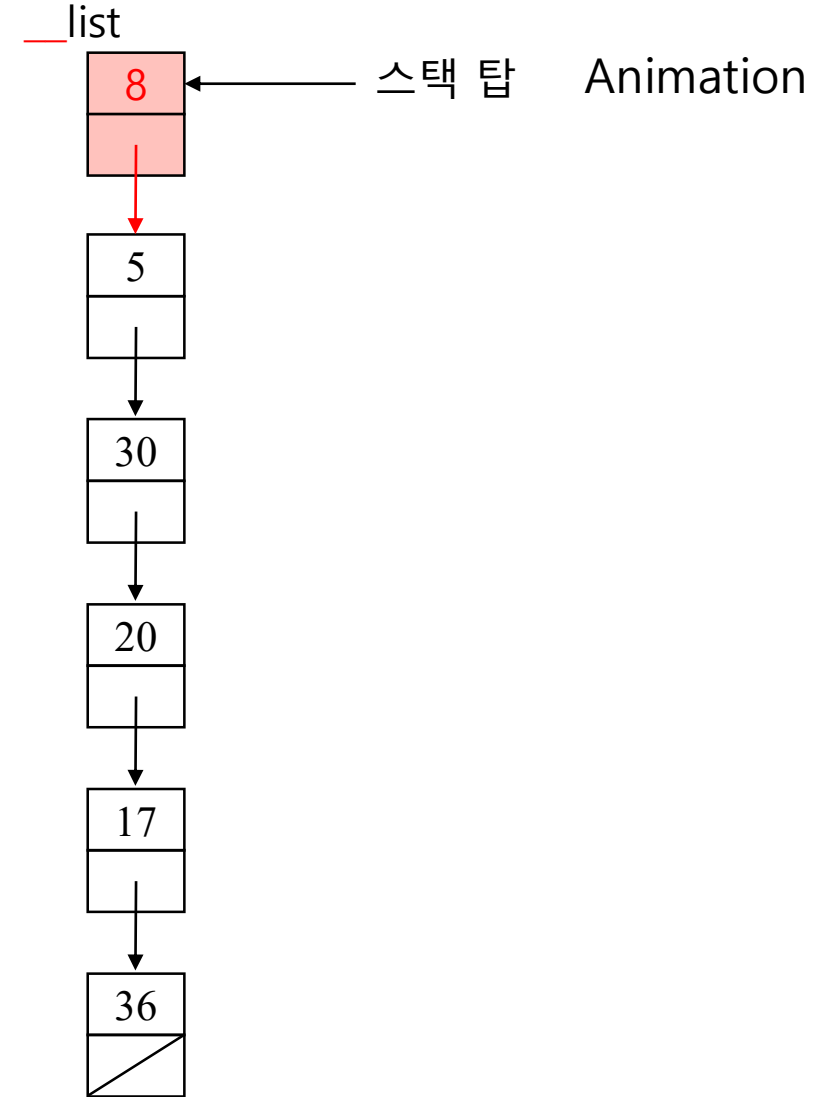
핵심 작업

```
def push(self, x):
    self.__list.insert(0, x)
```

연결 리스트로 구현된
스택의 원소 삽입



push(8)

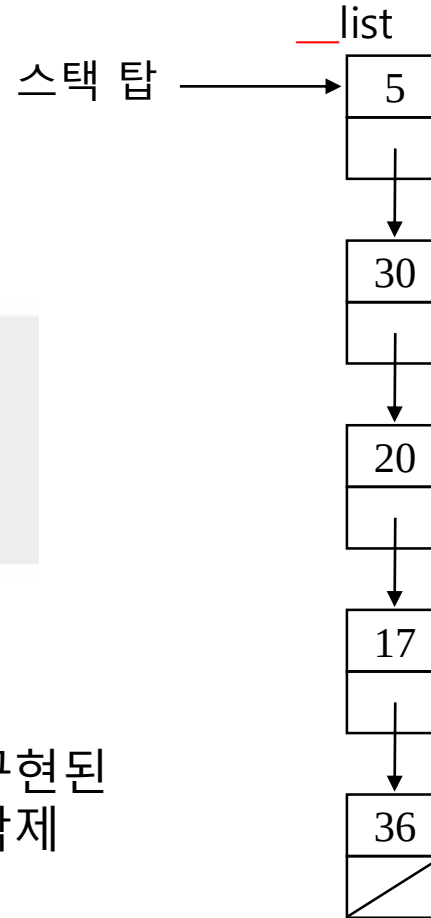


삭제(Deletion) : POP

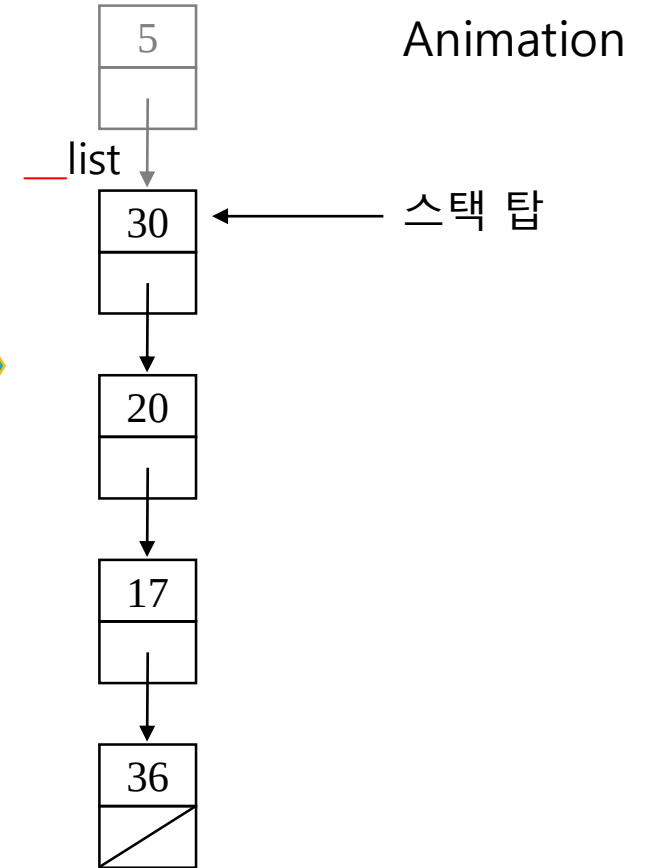
핵심 작업

```
def pop(self):  
    return self.__list.pop(0)
```

연결 리스트로 구현된
스택의 원소 삭제



pop()



기타 작업

top() `return __list.get(0)`

```
def top(self):  
    if self.isEmpty():  
        return None  
    else:  
        return self.__list.get(0)
```

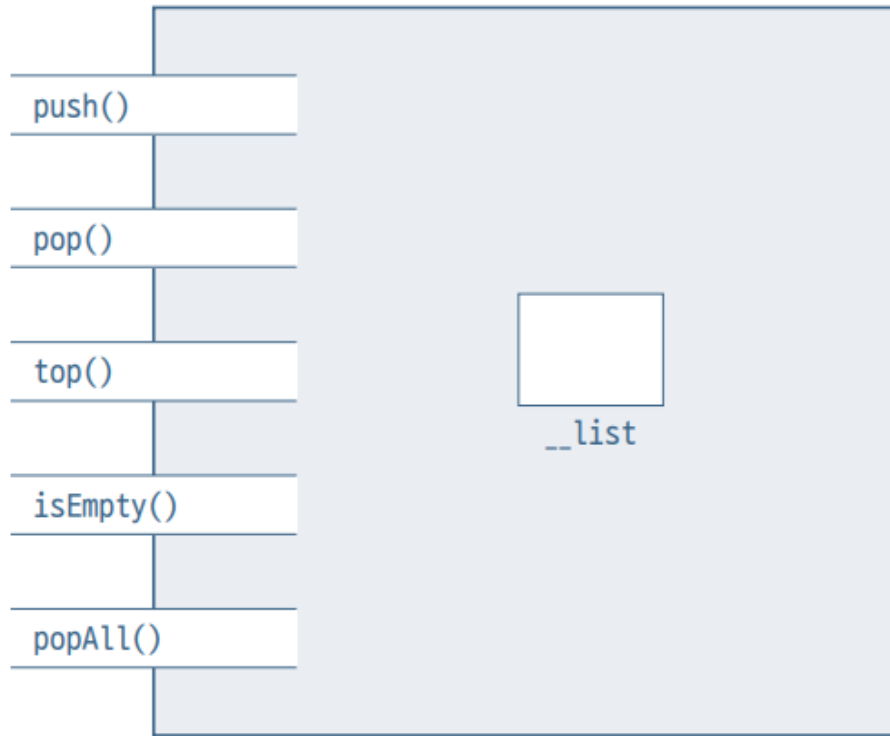
isEmpty() `return __list.isEmpty()`

```
def isEmpty(self) -> bool:  
    return self.__list.isEmpty()
```

popAll() `__list.clear()`

```
def popAll(self):  
    self.__list.clear()
```

파이썬 구현



연결 리스트 스택 객체 구조

```

class LinkedStack:
    def __init__(self):
        self.__list = LinkedListBasic()

    def push(self, newItem):
        self.__list.insert(0, newItem)

    def pop(self):
        return self.__list.pop(0)

    def top(self):
        return self.__list.get(0)

    def isEmpty(self) -> bool:
        return self.__list.isEmpty()

    def popAll(self):
        self.__list.clear()

    def printStack(self):
        print("Stack from top:", end=' ')
        for i in range(self.__list.size()):
            print(self.__list.get(i), end=' ')
        print()
    
```

연결 리스트 스택 사용 예

연결 리스트 스택 사용 예

```
st1 = linkedStack()
st1.push(100)
st1.push(200)
print("Top is", st1.top())
st1.pop()
st1.push('Monday')
st1.printStack()
print('isEmpty?', st1.isEmpty())
```

```
Top is 200
Stack from top:Monday100
isEmpty? False
```

수행 결과

```
Top is 200
Stack from top: Monday 100
isEmpty? False
```

문자열 뒤집기

```
def reverse(str):  
    st = ListStack()  
    for i in range(len(str)):  
        st.push(str[i])  
    out = ""  
    while not st.isEmpty():  
        out += st.pop()  
    return out  
  
def main():  
    input = "Test Seq 12345" # 테스트 입력 문자열  
    answer = reverse(input)  
    print("Input string: ", input)  
    print("Reversed string: ", answer)  
  
if __name__ == "__main__":  
    main()
```

```
Input string:  Test Seq 12345  
Reversed string:  54321 qeS tseT
```

main()과 __name__ 사용법

Postfix 계산

- 1 문자가 숫자이면 `push()`를 통해 스택에 삽입한다. 단, 바로 앞의 문자가 숫자이면 연속된 숫자로서 수 하나에 속하므로 스택 탑에 저장된 수를 뽑아서(`pop()`) 합쳐 계산한 다음 스택에 다시 넣어준다(`push()`). 예를 들어, 스택 탑에 25가 저장되어 있고 현재 숫자가 3이면 앞부분의 25는 253의 25이므로 $25 * 10 + 3$ 으로 계산해줘야 한다. 이런 식으로 수가 끝나는 공백을 만날 때까지 스택 탑에 있는 수를 뽑아서(`pop()`) 계산한 다음 결과를 스택에 넣어준다(`push()`).
- 2 문자가 연산자이면, 스택의 맨 위에 있는 수 2개를 뽑아서(`pop()` 2번) 계산한 다음 결과를 스택에 삽입한다(`push()`).
- 3 문자가 공백이면 그냥 무시하고 넘어간다.

Postfix 계산

```
from listStack import ListStack

def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i] # i번 문자. 번호는 0번부터
        if ch.isdigit(): # ch가 숫자
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else: # ch가 공백
            digitPreviously = False

    return s.pop()
```

Postfix 계산

```
def isOperator(ch) -> bool: # 연산자인가?
    return (ch == '+' or ch == '-' or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int: # 연산하기
    return {'+': opr1 + opr2, '-': opr1 - opr2, '*': opr1 * opr2, '/': opr1 // opr2}[ch]

def main():
    postfix = "700 3 47 + 6 * - 4 /" # 테스트 샘플 입력(후위 표현식)
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
Input string: 700 3 47 + 6 * - 4 /
Answer: 100
48 57
```

괄호 검사

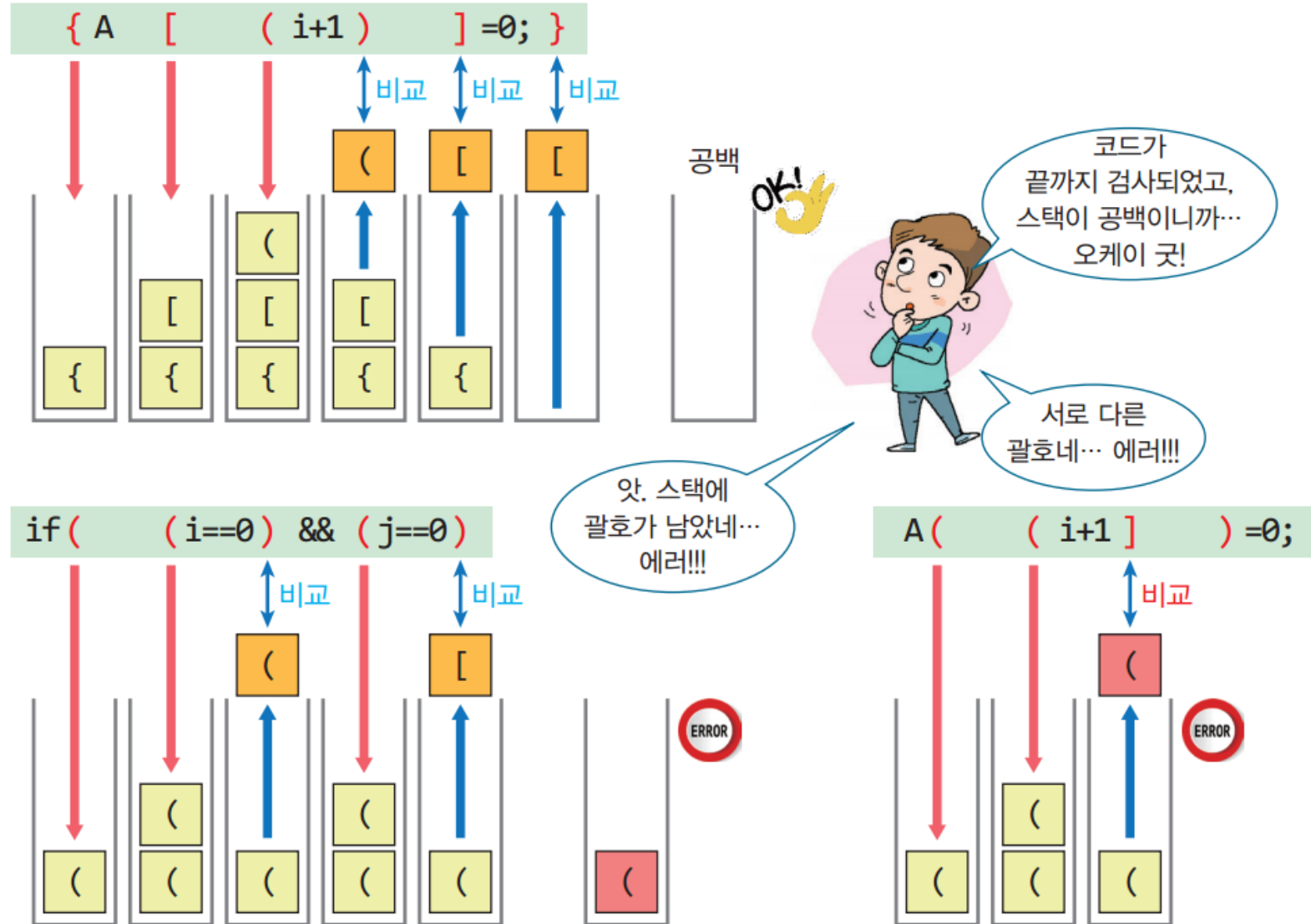
- 괄호의 종류: 대중소 ('[', ']'), ('{', '}'), ('(', ')')
 - 조건 1: 왼쪽 괄호의 개수와 오른쪽 괄호의 개수가 같아야 한다.
 - 조건 2: 같은 타입의 괄호에서 왼쪽 괄호가 오른쪽 괄호보다 먼저 나와야 한다.
 - 조건 3: 서로 다른 타입의 괄호 쌍이 서로를 교차하면 안 된다.
-
- 괄호 사용 예

<code>{ A[(i+1)]=0; }</code>	→ 오류 없음
<code>if ((i==0) && (j==0))</code>	→ 오류: 조건 1 위반
<code>while (it < 10) { it--; }</code>	→ 오류: 조건 2 위반
<code>A[(i+1)]=0;</code>	→ 오류: 조건 3 위반

괄호 검사 방법

- 문자를 저장하는 스택을 준비한다. 처음에는 공백 상태가 되어야 한다.
- 입력 문자열의 문자를 하나씩 읽어 왼쪽 괄호를 만나면 스택에 삽입한다.
- 오른쪽 괄호를 만나면 pop()연산으로 가장 최근에 삽입된 괄호를 꺼낸다. 이때 스택이 비었으면 조건 2에 위배된다.
- 꺼낸 괄호가 오른쪽 괄호와 짝이 맞지 않으면 조건 3에 위배된다.
- 끝까지 처리했는데 스택에 괄호가 남아 있으면 조건 1에 위배된다.

괄호 검사 예



괄호 검사 알고리즘

```
def checkBrackets(statement):  
    stack = Stack()  
    for ch in statement:                # 문자열의 각 문자에 대해  
        if ch in ('{', '[', '('):        # in '{(['도 동일하게 동작함  
            stack.push(ch)  
        elif ch in ('}', ']', ')'):      # in '}]})'도 동일하게 동작함  
            if stack.isEmpty() :  
                return False            # 조건 2 위반  
            else :  
                left = stack.pop()  
                if (ch == "}" and left != "{") or \    
                    (ch == "]" and left != "[") or \    
                    (ch == ")" and left != "(") :  
                    return False        # 조건 3 위반  
    return stack.isEmpty()              # False이면 조건 1 위반
```


괄호 검사 테스트

```
str = ( "{ A[(i+1)] = 0; }", "if( (i==0) && (j==0 )", "A[ (i+1] ) = 0;" )  
for s in str:  
    m = checkBrackets(s)  
    print(s, " ---> ", m)
```

```
{ A[(i+1)] = 0; } ---> True  
if( (i==0) && (j==0 ) ---> False  
A[ (i+1] ) = 0; ---> False
```


미로탐색

- 미로에서 출구를 찾는 것
- 시행 착오: 일단 하나의 경로를 선택하여 시도해 보고, 막히면 다시 다른 경로를 시도
- 다른 경로들을 어딘가에 저장해야 함
- DFS, Depth First Search
 - 가던 길이 막히면 가장 최근에 있었던 갈림길로 되돌아가서 다른 길을 찾는 방법 → 스택 사용



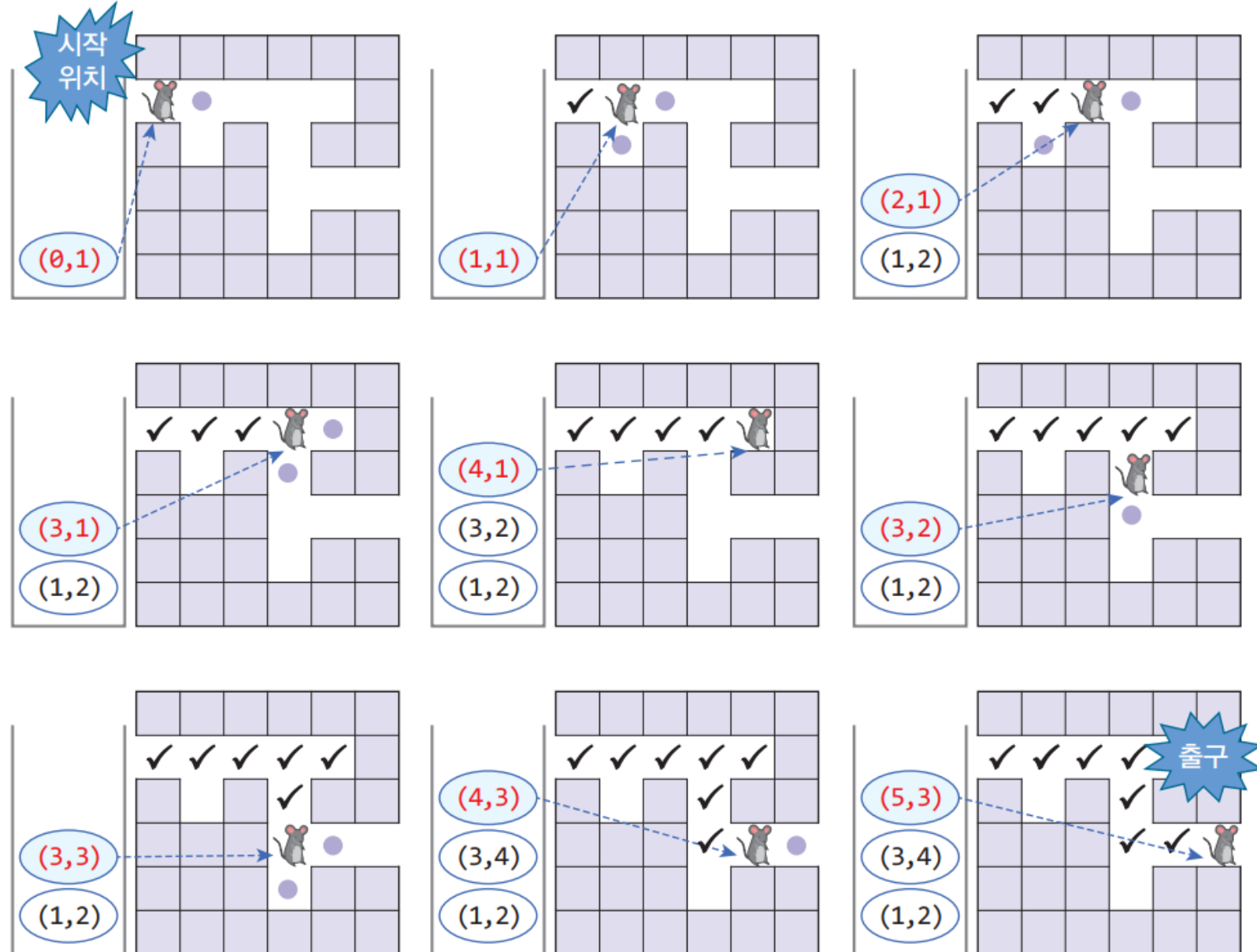
	0	1	2	3	4	5
0						
1						
2						
3						
4						
5						

```

map = [ [ '1', '1', '1', '1', '1', '1' ],
        [ 'e', '0', '0', '0', '0', '1' ],
        [ '1', '0', '1', '0', '1', '1' ],
        [ '1', '1', '1', '0', '0', 'x' ],
        [ '1', '1', '1', '0', '1', '1' ],
        [ '1', '1', '1', '1', '1', '1' ] ]
    
```

MAZE_SIZE = 6

미로 탐색



미로 탐색

```

def DFS() :
    print('DFS: ')
    stack = ArrayStack(100)
    stack.push((0,1))

    while not stack.isEmpty():
        here = stack.pop()
        print(here, end='->')
        (x,y) = here

        if (map[y][x] == 'x') :
            return True
    
```

```

else :
    map[y][x] = '.' # 현재 위치는 방문함. '.'표시
    if isValidPos(x, y - 1): stack.push((x, y - 1)) # 상
    if isValidPos(x, y + 1): stack.push((x, y + 1)) # 하
    if isValidPos(x - 1, y): stack.push((x - 1, y)) # 좌
    if isValidPos(x + 1, y): stack.push((x + 1, y)) # 우
    print(' 현재 스택: ', stack)

    return False # 탐색 실패
    
```

STACK 응용

```
class Stack :
    def __init__(self):
        self.top = []

    def isEmpty(self):
        return len(self.top) == 0

    def size(self):
        return len(self.top)

    def clear(self):
        self.top = []

    def push(self, item):
        self.top.append(item)

    def pop(self):
        if not self.isEmpty():
            return self.top.pop(-1)

    def peek(self):
        if not self.isEmpty():
            return self.top[-1]

    def __str__(self):
        return str(self.top)
```

```
map = [['1','1','1','1','1','1'],
        ['e','0','0','0','0','1'],
        ['1','0','1','0','1','1'],
        ['1','1','1','0','0','x'],
        ['1','1','1','0','1','1'],
        ['1','1','1','1','1','1']]
MAZE_SIZE = 6

def isValidpos(x, y):
    if x < 0 or y < 0 or x >= MAZE_SIZE or y >= MAZE_SIZE:
        return False
    else:
        return map[y][x] == '0' or map[y][x] == 'x'
```

미로 탐색 소스

```
def DFS():
    stack = Stack()
    stack.push((0,1))
    print('DFS: ')

    while not stack.isEmpty():
        here = stack.pop()
        print(here, end = '->')
        (x,y) = here
        if (map[y][x] == 'x'):
            return True
        else:
            map[y][x] = '.'

            if isValidpos(x, y-1):
                stack.push((x, y-1))
            if isValidpos(x, y+1):
                stack.push((x, y+1))
            if isValidpos(x-1, y):
                stack.push((x-1, y))
            if isValidpos(x+1, y):
                stack.push((x+1, y))
        print(' 현재 스택 : ', stack)
    return False
```

미로 탐색 테스트

```
result = DFS()  
if result : print(' --> 미로탐색 성공')  
else : print(' --> 미로탐색 실패')
```

DFS:

```
(0, 1)-> 현재 스택 : [(1, 1)]  
(1, 1)-> 현재 스택 : [(1, 2), (2, 1)]  
(2, 1)-> 현재 스택 : [(1, 2), (3, 1)]  
(3, 1)-> 현재 스택 : [(1, 2), (3, 2), (4, 1)]  
(4, 1)-> 현재 스택 : [(1, 2), (3, 2)]  
(3, 2)-> 현재 스택 : [(1, 2), (3, 3)]  
(3, 3)-> 현재 스택 : [(1, 2), (3, 4), (4, 3)]  
(4, 3)-> 현재 스택 : [(1, 2), (3, 4), (5, 3)]  
(5, 3)->--> 미로 탐색 성공
```

파이썬에서 스택 사용하기

방법 1) Stack 클래스를 구현해서 사용하기

방법 2) 파이썬 리스트를 스택으로 사용하기

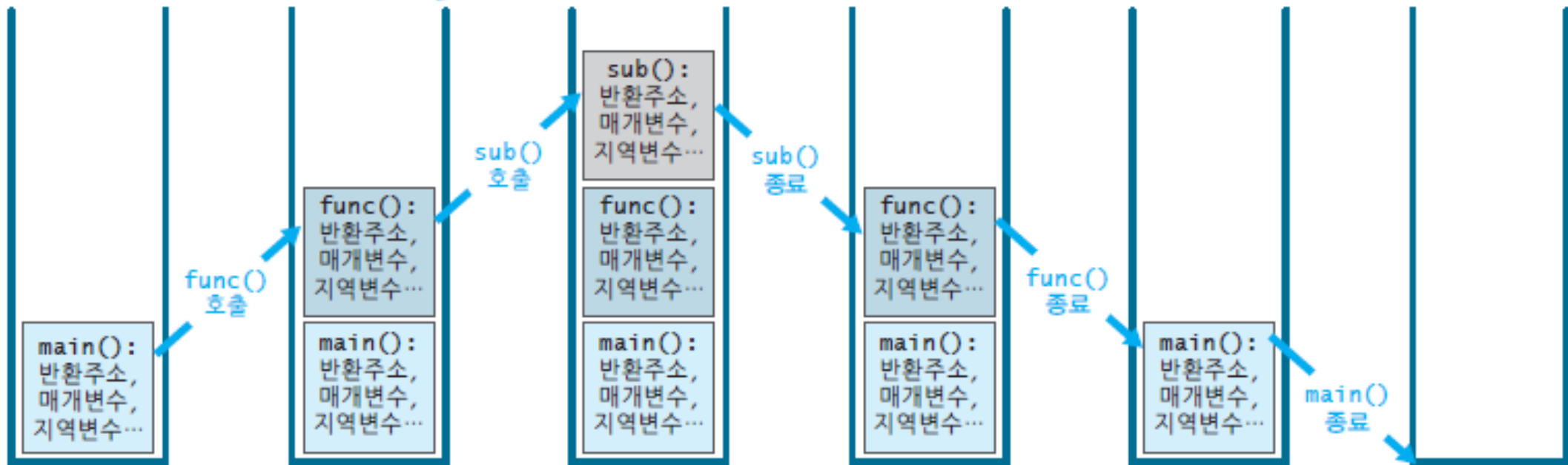
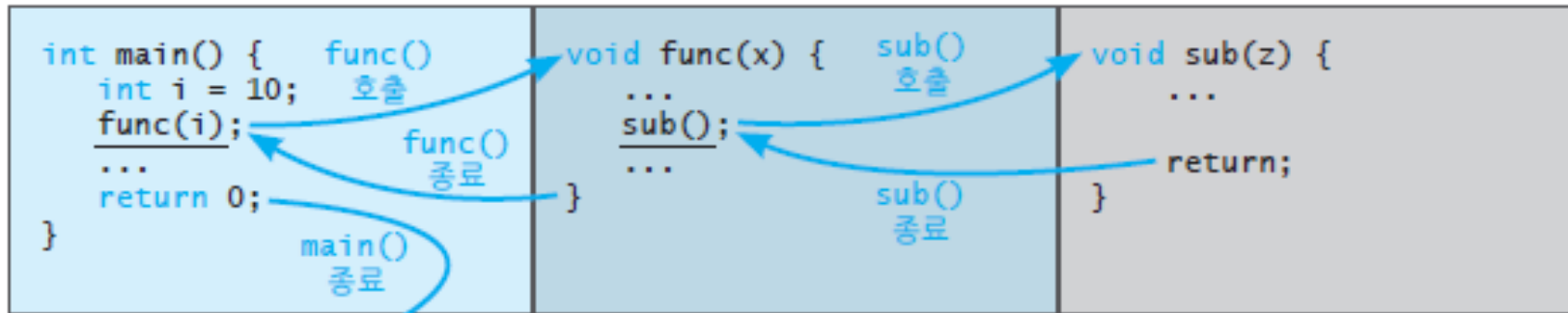
연산	ArrayStack	파이썬 list	사용 예
삽입	push()	메서드 append() 사용	L.append(e)
삭제	pop()	메서드 pop() 사용	L.pop()
요소의 수	size()	내장 함수 len()을 사용	len(L)
공백 상태 검사	isEmpty()	요소의 수가 0인지 검사	len(L) == 0
포화 상태 검사	isFull()	list는 용량이 무한대라 포화 상태는 의미가 없음. 항상 False	False
상단 들여다보기	peek()	맨 마지막 요소 참조	L[len(L)-1] 또는 L[-1]

방법 3) queue 모듈의 LifoQueue 사용하기

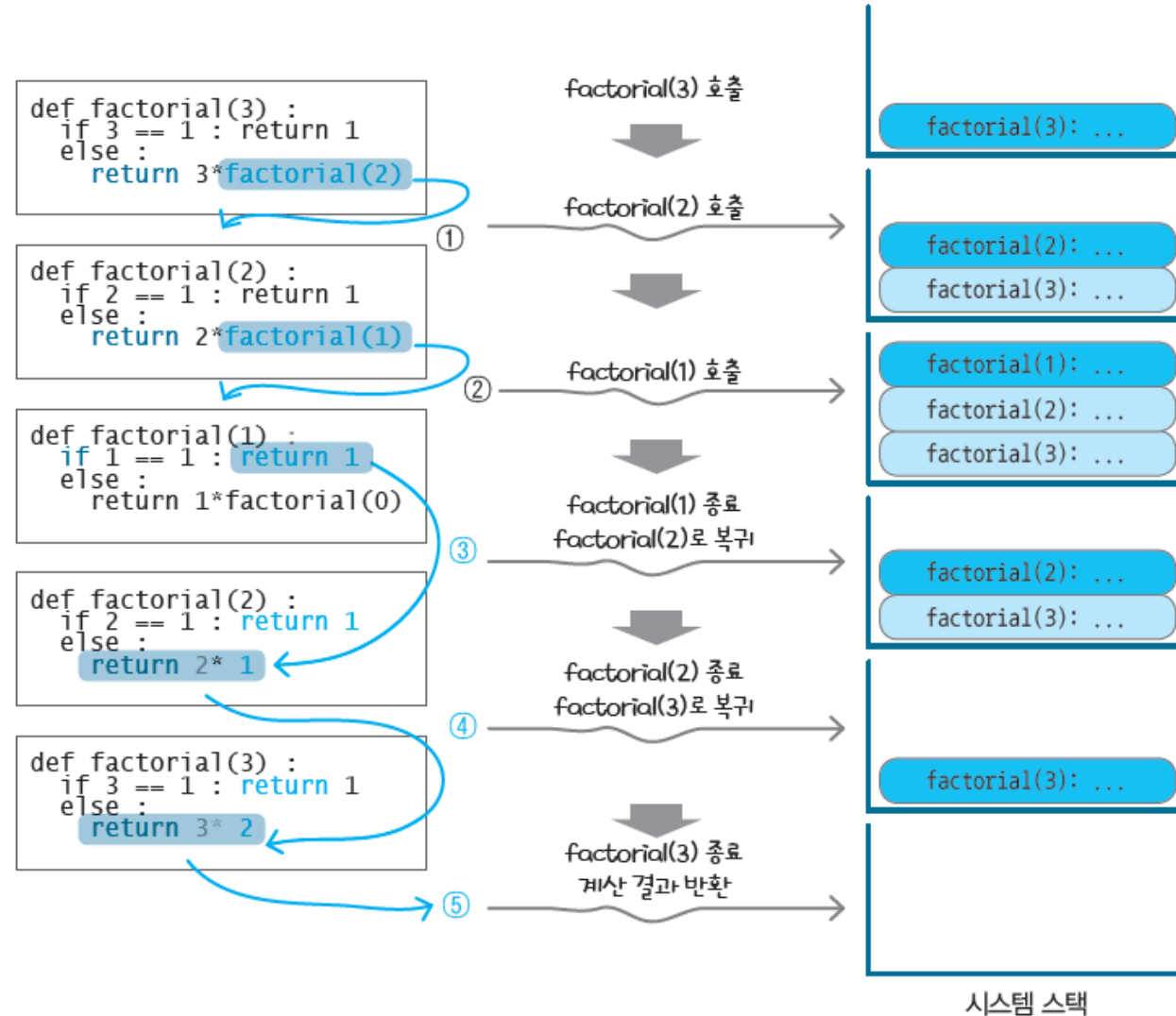
- `import queue` # 파이썬의 큐 모듈 포함
- `s = queue.LifoQueue(maxsize=20)` # 스택 객체 생성(크기 20)

연산	ArrayStack	queue.LifoQueue
삽입/삭제	<code>push()</code> , <code>pop()</code>	<code>put()</code> , <code>get()</code>
공백/포화 상태 검사	<code>isEmpty()</code> , <code>isFull()</code>	<code>empty()</code> , <code>full()</code>
상단 들여다보기	<code>peek()</code>	제공하지 않음

함수의 호출과 반환을 위한 시스템 스택 예



순환적인 팩토리얼 함수 동작의 이해



Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 강민, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- 자료구조와 알고리즘 with 파이썬, 최영규, 생능북스